

Горчак Дмитрий, 212. Git – локальная система контроля версий: назначение, команды, встроенная справка

Система контроля версий хранит историю проекта: не последнюю версию каждого файла, а все состояния, через которые проект прошёл, вместе с объяснением, зачем каждое из них появилось. Git (произносится «гит») – самая распространённая из таких систем; его особенность в том, что вся история лежит на компьютере разработчика и работать с ней можно без сети.

Ниже разобрано, как git хранит проект, какими командами делается повседневная работа, как спрашивать встроенную справку и что означают опции команд.

Как git хранит проект

Репозиторий (repository, хранилище) – это вся история проекта. Он лежит в скрытой папке .git рядом с файлами и никуда больше не обращается.

Коммит (commit, фиксация) – одно сохранённое состояние всех файлов сразу, а не отдельный изменённый файл. Вместе с файлами коммит хранит автора, дату, сообщение и ссылку на предыдущий коммит; из этих ссылок и складывается история.

Имя коммита – хеш (hash), короткая строка вроде 33d77fd, посчитанная по его содержимому. Если изменится хотя бы один знак в любом файле, хеш будет другим, поэтому незаметно подменить сохранённую версию нельзя.

Ветка (branch) – подвижный указатель на коммит. Она сдвигается вперёд при каждом сохранении, поэтому завести ветку почти ничего не стоит. HEAD – указатель на ту ветку, в которой вы сейчас работаете.

Тег (tag, метка) – такой же указатель, только неподвижный. Он закрепляет конфигурацию, то есть состав версий всех файлов, из которых собрана программа: по метке её можно собрать заново ровно такой, какой она была в нужный момент.

Три состояния файла

Правка проходит три места, и почти все команды занимаются переносом изменений между ними:

- 1) рабочий каталог (working directory) – файлы, открытые в редакторе; здесь изменение ничем не защищено;
- 2) индекс (index, staging area – подготовленная область) – то, что отобрано для следующего сохранения командой git add;
- 3) репозиторий (.git) – сохранённые версии; изменения переносит туда команда git commit.

```
[рабочий каталог] --git add--> [индекс] --git commit--> [репозиторий .git]
[рабочий каталог] <--git restore-- [индекс] <--git restore --staged-- [.git]
```

Две ступени вместо одной нужны, чтобы сохранить одну правку, а вторую доделать позже: в индекс кладут только готовое.

Установка и настройка

В MSYS2 (среда с Linux-подобным терминалом для Windows) git ставится менеджером пакетов pacman:

```
$ pacman -S git
$ git --version
git version 2.55.0
```

В Linux вместо этого используется свой менеджер пакетов (apt, dnf, pacman), в Windows – установщик с сайта git-scm.com.

git config – настройки. Имя и почта обязательны: они попадают в каждый коммит.

```
$ git config --global user.name "Gorchak Dmitriy"
$ git config --global user.email "почта@пример"
```

Ключ --global означает «для всех моих репозиториях». Есть ещё --system (для всей машины) и --local (только для текущего репозитория); более узкий уровень перекрывает более широкий.

```
$ git config --list --local
core.autocrlf=false
user.name=Gorchak Dmitriy
```

core.autocrlf – настройка для Windows. Конец строки в Windows и в Linux записывается разными невидимыми символами (CRLF и LF), и без этой настройки git может показать изменённым весь файл, хотя поменялась одна строка.

Начало работы: init, status, add, commit

git init – создать репозиторий в текущей папке. Ключ -b (branch) задаёт имя основной ветки:

```
$ git init -b main
Initialized empty Git repository in /home/user/cones/.git/
```

git status – что изменено и что уже отобрано для сохранения. Эту команду полезно выполнять перед каждым шагом: она же подсказывает, какой командой сделать следующий.

```
$ git status -s
M data/cones.txt
```

Ключ -s (short, коротко) сокращает вывод до одной строки на файл: М – файл изменён (modified), А – добавлен (added), ?? – git про него ещё не знает.

git add – положить изменения в индекс:

```
$ git add data/cones.txt      # один файл
$ git add .                  # всё в текущей папке
```

git commit – сохранить содержимое индекса как новую версию. Ключ -m (message) задаёт сообщение прямо в командной строке, иначе откроется текстовый редактор:

```
$ git commit -m "Класс Point: точка на плоскости"
[main e117716] Класс Point: точка на плоскости
2 files changed, 96 insertions(+)
```

В ответе видно ветку (main), хеш нового коммита (e117716) и сколько файлов и строк изменилось.

Просмотр истории: log, show, diff

git log – список коммитов, от новых к старым. Без опций выводится подробно, по несколько строк на коммит; читать удобнее сокращённый вид:

```
$ git log --oneline --graph --decorate
* 33d77fd (HEAD -> main, tag: stage-19-09) Скриншот gnuplot
* ee7a7df Программа cones_app и сборка
* dc8a93d Чтение свойств из файла
```

Опции здесь такие: --oneline – одна строка на коммит, --graph – рисунок ветвлений слева, --decorate – имена веток и меток.

git show – что именно изменилось в конкретном коммите:

```
$ git show 33d77fd
```

git diff – сравнение. Без опций показывает правки, которых ещё нет в индексе; с ключом --staged – то, что войдёт в следующий коммит:

```
$ git diff
@@ -25,3 +25,4 @@
    10.0  10.0  -1.0   5.0
+   8.0  17.0   1.7   5.5
```

Строка со знаком + добавлена, со знаком – удалена, остальные показаны для понимания места.

Возврат к прошлой версии

git restore – вернуть файл к последней сохранённой версии, отменив правки в рабочем каталоге:

```
$ git restore data/cones.txt
```

Тот же git restore с ключом --staged убирает файл из индекса, не трогая сам файл:

```
$ git restore --staged data/cones.txt
```

git checkout – перейти к состоянию другого коммита или ветки; в новых версиях git для перехода между ветками есть более понятная команда git switch.

Ветки, слияние, метки

Ветка нужна, когда отдельную задачу хочется делать, не трогая основную работу. git switch -c (create) создаёт ветку и сразу переходит в неё:

```
$ git switch -c stl-list
$ git commit -am "Двухнаправленный список STL"
$ git switch main
```

git merge – слияние: git берёт изменения из указанной ветки и добавляет их в текущую. Ключ --no-ff (no fast-forward) заставляет создать отдельный коммит слияния, чтобы в истории осталось видно, что это была отдельная задача:

```
$ git merge --no-ff stl-list
```

Если обе ветки меняли одно и то же место файла, git сообщает о конфликте и отмечает спорные строки прямо в файле; нужно оставить правильный вариант и сделать коммит.

git tag – поставить метку на текущее состояние. Ключ -a (annotated) создаёт метку с автором и пояснением:

```
$ git tag -a stage-19-09 -m "Состояние к сроку 19.09"
$ git tag
stage-19-09
```

Встроенная справка

Справка лежит внутри самой программы, и спросить её обычно быстрее, чем искать ответ в интернете. Форм вызова три:

- 1) git help КОМАНДА – полное руководство по команде;
- 2) git КОМАНДА --help – то же самое руководство;
- 3) git КОМАНДА -h – короткая справка на несколько строк, только список опций.

```
$ git commit -h
usage: git commit [-a | --interactive | --patch] [-s] [-v]
      -m, --message <сообщение>    сообщение коммита
```

git help -a (all) выводит список всех команд, git help -g (guides) – список руководств по темам. Дальше открывается нужное: git help glossary объясняет понятия, git help revisions – как ссылаться на коммиты, git help everyday – типовые приёмы работы.

```
$ git help -a | head -8
$ git help glossary
```

Руководство открывается прямо в окне терминала: листается пробелом и стрелками, выход – клавиша q.

Опции

Опция – уточнение к команде. Короткие опции пишутся одной буквой после дефиса, длинные – словом после двух дефисов; у многих есть обе формы: -m и --message, -q и --quiet (тихо, без лишнего вывода).

Короткие опции можно склеивать: git commit -am это то же, что git commit -a -m, где -a (all) означает «взять все изменённые файлы, которые git уже отслеживает».

Разделитель из двух дефисов отделяет опции и имена веток от имён файлов – он нужен, когда ветка и файл называются одинаково:

```
$ git restore -- data/cones.txt
$ git log -3 -- data/cones.txt    # три последние правки файла
```

Часто используемые сочетания: git add -p (patch) – просмотреть изменения по кускам и отобрать нужные; git diff --staged – увидеть ровно то, что попадёт в коммит; git commit --amend (исправить) – переписать последний коммит, пока он никуда не отправлен.

Что не хранят в репозитории: .gitignore

В репозиторий кладут исходные тексты, данные и описание сборки, но не то, что компилятор и сама программа создают заново. Правило простое: если файл можно получить из того, что уже лежит в репозитории, хранить его не надо – иначе история растёт, а пользы нет.

Имена и шаблоны таких файлов перечисляют в файле .gitignore, и git перестаёт о них сообщать:

```
# результаты сборки
build/
bin/
*.o
*.exe

# файлы, которые создаёт сама программа
cones_plan.txt
cones_field.txt

# рисунки в корне проекта
/*.png
```

Звёздочка заменяет любую часть имени, косая черта в конце означает папку, а косая черта в начале – «только в корне проекта».

Пример истории небольшого проекта

Нижеследующая история учебного проекта cones: программа читает из файла параметры конусов, считает площадь основания и объём и строит поле фигур в gnuplot. Каждый законченный шаг сохранялся отдельным коммитом, поэтому видно, в каком порядке собиралась программа.

```
* 43a246a (HEAD -> main) Слияние ветки stl-list: работа со списком
|\
| * d5efe6d (stl-list) Двухнаправленный список STL: сортировка
|/
* 33d77fd (tag: stage-19-09) Скрипт gnuplot: вид сверху и ландшафт
* eeefa7df Программа cones_app и сборка: build.sh и CMakeLists.txt
* dc8a93d Чтение свойств из файла, таблица и данные для gnuplot
* 27a018e Класс Cone: конус (x, y, r, h), объём  $V = S \cdot h / 3$ 
* b8af7c1 Класс Circle: круг и площадь как вычисляемый атрибут
* e117716 Класс Point: точка на плоскости, поля private
* f4dfd46 Начало работы: README и .gitignore
```

История читается снизу вверх. Работа над списком шла в отдельной ветке stl-list и влила слиянием с ключом --no-ff – отсюда развилка в рисунке. На коммит 33d77fd поставлена метка stage-19-09.

Обычная правка занимает четыре команды: посмотреть, что изменилось, посмотреть, как именно, отобразить и сохранить.

```
$ git status -s
$ git diff
$ git add data/cones.txt
$ git commit -m "Ещё один конус в исходных данных"
```

Частые ошибки

- 1) забыть `git add` – коммит уходит без изменений; помогает привычка смотреть `git status` перед каждым сохранением;
- 2) сложить в один коммит несколько несвязанных правок – потом не понять, какая из них сломала сборку;
- 3) сообщения вида «fix» и «работает» – через неделю по ним ничего не восстановить; в сообщении пишут, что и зачем изменено;
- 4) хранить собранные программы и рисунки – репозиторий растёт, а смысла нет;
- 5) не настроить `core.autocrlf` на Windows – тогда `git` показывает изменённым весь файл вместо одной строки.

Копия в облаке

Облачный репозиторий (GitHub, GitLab и подобные) – это ещё одна копия того же самого `.git`. Локальные команды при переходе к нему не меняются, добавляются только две:

```
$ git remote add origin АДРЕС_РЕПОЗИТОРИЯ
$ git push -u origin main
```

`git remote` – список и настройка адресов копий (`origin` – принятое имя основной копии), `git push` – отправить туда уже сделанные коммиты, `git pull` – забрать чужие изменения себе.

Заключение

Git хранит не файлы, а историю состояний проекта и позволяет вернуться в любое из них. Повседневная работа сводится к десятку команд: `status`, `add`, `commit`, `log`, `diff`, `restore`, `switch`, `merge`. Точное значение любой опции всегда можно уточнить встроенной справкой, не выходя из терминала.

```
[правка файла] -(git add)-> [индекс] -(git commit)-> [история в .git]
[история] -(git log, git show, git diff)-> [что и когда изменилось]
[история] -(git restore, git switch)-> [возврат к нужному состоянию]
[история] -(git push)-> [копия в облаке]
```

Приложение. Памятка по командам git

Цель	Команда	Пример	Описание
Создать репозиторий	<code>git init</code>	<code>git init -b main</code>	создаёт папку <code>.git</code> ; <code>-b</code> задаёт имя ветки
Посмотреть состояние	<code>git status</code>	<code>git status -s</code>	что изменено и что отобрано; <code>-s</code> – коротко
Отобразить изменения	<code>git add</code>	<code>git add data/cones.txt</code>	переносит правки в индекс
Сохранить версию	<code>git commit</code>	<code>git commit -m "текст"</code>	фиксирует индекс; <code>-m</code> – сообщение
Сохранить всё сразу	<code>git commit -am</code>	<code>git commit -am "текст"</code>	<code>add</code> и <code>commit</code> для известных файлов
Посмотреть правки	<code>git diff</code>	<code>git diff --staged</code>	различия; <code>--staged</code> – то, что войдёт в коммит
Посмотреть историю	<code>git log</code>	<code>git log --oneline --graph</code>	список коммитов; <code>--oneline</code> – по строке
Посмотреть коммит	<code>git show</code>	<code>git show 33d77fd</code>	что изменилось в конкретном коммите

Отменить правку	<code>git restore</code>	<code>git restore file.txt</code>	возвращает файл к сохранённой версии
Убрать из индекса	<code>git restore --staged</code>	<code>git restore --staged file.txt</code>	снимает отбор, файл не трогает
Создать ветку	<code>git switch -c</code>	<code>git switch -c stl-list</code>	создаёт ветку и переходит в неё
Слить ветку	<code>git merge</code>	<code>git merge --no-ff stl-list</code>	переносит изменения в текущую ветку
Поставить метку	<code>git tag -a</code>	<code>git tag -a stage-19-09 -m "..."</code>	закрепляет состояние проекта
Удалить файл	<code>git rm</code>	<code>git rm old.cpp</code>	удаляет файл и запоминает это в истории
Переименовать файл	<code>git mv</code>	<code>git mv a.cpp b.cpp</code>	переименование с сохранением истории
Справка по команде	<code>git help</code>	<code>git help commit</code>	полное руководство; <code>-h</code> — кратко
Адрес копии в облаке	<code>git remote add</code>	<code>git remote add origin АДРЕС</code>	привязывает облачный репозиторий
Отправить в облако	<code>git push</code>	<code>git push -u origin main</code>	переносит коммиты в облачную копию